

E-commerce Project Development Plan

Project Overview

The e-commerce platform will combine modern design aesthetics with powerful functionality, featuring:

- Customer-facing storefront with artistic design elements
- Feature-rich user accounts and order management
- Comprehensive admin dashboard for complete business control
- Marketing and promotional tools including collections, banners, and coupons
- Robust reporting and analytics capabilities

Technical Architecture

Technology Stack

Component	Technology
Frontend	React, Javascript, Tailwind CSS
State Management	Redux Toolkit or React Query
Backend	Node.js/Express or NestJS
Database	PostgreSQL with Prisma ORM
Authentication	JWT-based auth system
Caching	Redis
Media Storage	Cloudinary
Frontend Hosting	Cloudflare Pages
Backend Hosting	AWS EC2 (Mumbai region)
CI/CD	GitHub Actions or GitLab CI
Monitoring	AWS CloudWatch + Sentry

Infrastructure Deployment

- **Frontend:** Cloudflare Pages for global CDN distribution
- **Backend:** AWS EC2 t3.medium instance (2 vCPU, 4GB RAM)

- **Database:** Self-managed PostgreSQL on EC2 or AWS RDS
- **Caching:** Redis on EC2
- **Media Assets:** Cloudinary CDN
- **Monitoring:** Combination of AWS CloudWatch and application monitoring

API Development Requirements

A total of 77 distinct API endpoints will be required across 17 categories:

Core Functionality APIs (28 endpoints)

- **Authentication** (5): Registration, login, password reset, admin authentication, token refresh
- **Product Management** (6): Listing, details, search, categories, reviews, featured products
- **Shopping Cart** (4): Cart management, retrieval, save for later, cart merging
- **Order Management** (6): Creation, history, details, status updates, cancellation, returns
- **Payment Processing** (4): Payment intent, verification, refunds, invoicing
- **User Profile** (3): Profile management, address management, wishlist

Marketing & Engagement APIs (26 endpoints)

- **Collection Management** (5): Create, update, delete, list, and get collection details
- **Banner Management** (4): Create, update, delete, and list promotional banners
- **Notification System** (4): Create templates, send notifications, retrieve user notifications, update status
- **Email Campaign** (6): Create/update templates, send bulk emails, scheduling, analytics, subscriber management
- **Coupon & Discounts** (7): Create, update, delete, validate, apply coupons, list active offers, analytics

Admin & Support APIs (19 endpoints)

- **Admin Dashboard** (5): Sales analytics, inventory, user management, order management, content management
- **Reviews & Ratings** (4): Submit, moderate, list, delete reviews
- **Customer Support** (3): Create tickets, update status, list tickets
- **Analytics & Reporting** (4): Sales reports, product performance, customer analytics, inventory reports
- **Content Management** (3): Update site content, manage SEO settings, blog post management

Miscellaneous APIs (4 endpoints)

- Image upload (Cloudinary integration)
- Shipping calculation

- Notification preferences
- Product recommendations

Development Phases

Phase 1: Foundation

- Project setup and environment configuration
- Database schema design and implementation
- Authentication system development
- Basic product catalog functionality
- Core API structure implementation

Phase 2: Core Functionality

- Product management system
- Shopping cart implementation
- Checkout and payment processing
- Order management system
- User profile and address management

Phase 3: Admin Capabilities

- Admin dashboard implementation
- Inventory management system
- Order processing workflows
- User management tools
- Basic reporting and analytics

Phase 4: Marketing & Engagement

- Collection and banner management
- Coupon and discount system
- Email marketing integration
- Notification system
- Review and rating functionality

Phase 5: Refinement & Launch

- Performance optimization
- Security hardening
- Comprehensive testing
- Deployment preparation
- Launch and initial monitoring

Deployment Plan

Infrastructure Setup

1. Network Configuration

- Set up Virtual Private Cloud (VPC) with private and public subnets
- Configure security groups and network ACLs
- Implement bastion host for secure administrative access

2. Environment Provisioning

- Create separate development, staging, and production environments
- Implement infrastructure-as-code using Terraform or AWS CloudFormation
- Configure load balancers and auto-scaling groups for production

3. Database Deployment

- Set up primary PostgreSQL instance with read replicas for scaling
- Configure connection pooling with PgBouncer
- Implement database encryption at rest and in transit

4. Frontend Deployment

- Set up Cloudflare Pages project with environment variables
- Configure build pipelines with cache optimization
- Set up custom domain with SSL certification

5. Backend Deployment

- Deploy Node.js/Express application to EC2 instances
- Configure PM2 or similar process manager for application management
- Set up Redis cluster for caching and session management

CI/CD Pipeline

1. Code Repository Structure

- Organize monorepo or multi-repo structure with clear separation of concerns
- Implement branch protection and code review requirements
- Set up automated linting and code quality checks

2. Continuous Integration

- Configure automated testing for each pull request
- Implement build artifacts versioning
- Set up dependency vulnerability scanning

3. Continuous Deployment

- Create deployment workflow for each environment
- Implement blue-green deployment strategy for zero-downtime updates
- Configure automatic rollback mechanisms for failed deployments

4. Monitoring Integration

- Set up application and infrastructure monitoring alerts

- Configure error tracking and reporting
- Implement request tracing for performance optimization

Backup & Disaster Recovery

Database Backup Strategy

1. Regular Automated Backups

- Daily full database backups with transaction log backup every hour
- Store backups in multiple geographic regions
- Maintain backup retention policy (daily for 7 days, weekly for 1 month, monthly for 1 year)

2. Point-in-Time Recovery

- Configure continuous archiving and WAL (Write-Ahead Logging)
- Test recovery process monthly to ensure reliability
- Document recovery procedures with step-by-step instructions

3. Data Validation

- Implement checksum verification for backup integrity
- Perform periodic restoration tests to validate backup quality
- Monitor backup success/failure and storage capacity

Disaster Recovery Plan

1. Recovery Time Objective (RTO) & Recovery Point Objective (RPO)

- Define acceptable data loss timeframe (RPO)
- Establish maximum acceptable downtime (RTO)
- Document service level agreements for recovery

2. Infrastructure Redundancy

- Implement multi-availability zone deployment
- Configure auto-scaling for handling instance failures
- Set up database failover mechanisms

3. Failover Procedures

- Document manual failover steps for critical components
- Create automated recovery scripts where possible
- Establish communication protocols during outages

4. Regular Testing

- Schedule quarterly disaster recovery drills
- Simulate various failure scenarios (instance, database, network)
- Review and improve recovery procedures based on test results

Business Continuity

1. Documentation

- Maintain updated system architecture diagrams
- Document all configuration settings and dependencies
- Create comprehensive runbooks for operational procedures

2. Knowledge Transfer

- Cross-train team members on recovery procedures
- Establish escalation paths for different types of failures
- Create incident response playbooks

Development Considerations

Security Requirements

- Implement HTTPS across all connections
- PCI-DSS compliance for payment processing
- Data encryption for sensitive information
- OWASP best practices implementation
- Regular security audits and penetration testing

Performance Optimization

- Implement efficient caching strategies with Redis
- Content delivery optimization via CDN
- Database query optimization
- Server-side rendering for critical pages
- Image optimization via Cloudinary

Scalability Planning

- Horizontal scaling capability for backend services
- Database read replicas for heavy traffic periods
- Caching strategy for high-demand products
- Microservice architecture consideration for future expansion

Quality Assurance

- Comprehensive unit testing (Jest)
- Integration testing for API endpoints
- End-to-end testing (Cypress)
- User acceptance testing
- Performance and load testing

Potential Open Source Tools for Dashboard Development

To accelerate development of the admin dashboard, consider these open source solutions:

1. **Refine** - React-based framework specifically for admin panels with ready-to-use data providers
2. **React Admin** - Frontend framework built on Material Design for B2B applications
3. **Appsmith** - Open-source framework for building dashboards and internal tools
4. **Retool** - Low-code platform with free tier for internal tools